

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO  
FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN  
INGENIERÍA EN COMPUTACIÓN

VINCULACIÓN EMPRESARIAL  
DOCUMENTACIÓN DEL PROYECTO

# EDUINSPECT - SISTEMA DE GESTIÓN DE INCIDENCIAS

ALUMNO: RAMÍREZ ALCÁNTARA PEDRO ANTONIO

PROFESOR: AARÓN VELASCO AGUSTÍN  
GRUPO: 2007  
SEMESTRE: 2026-2

MÉXICO, CDMX. MAYO 2026

## PUNTO 2: ÍNDICE

### ÍNDICE

|                                     |    |
|-------------------------------------|----|
| 1. Portada .....                    | 1  |
| 2. Índice .....                     | 2  |
| 3. Resumen Ejecutivo .....          | 3  |
| 4. Introducción .....               | 4  |
| 5. Planteamiento del Problema ..... | 6  |
| 6. Objetivos .....                  | 9  |
| 7. Marco Teórico .....              | 11 |
| 8. Alcance del Sistema .....        | 16 |
| 9. Requerimientos del Sistema ..... | 18 |
| 10. Casos de Uso .....              | 21 |
| 11. Historias de Usuario .....      | 28 |
| 12. Tecnologías Seleccionadas ..... | 30 |

|                                              |    |
|----------------------------------------------|----|
| 13. Arquitectura del Sistema .....           | 33 |
| 14. Estructura del Proyecto .....            | 35 |
| 15. Uso de Contenedores y Despliegue .....   | 37 |
| 16. Modelo de Base de Datos .....            | 39 |
| 17. Estrategia de Control de Versiones ..... | 42 |
| 20. Riesgos del Proyecto .....               | 44 |
| 21. Conclusiones .....                       | 46 |
| 22. Referencias .....                        | 48 |
| ...                                          |    |

---

## PUNTO 3: RESUMEN EJECUTIVO

### Problema identificado:

En las instalaciones de la FES Aragón - UNAM, el proceso de gestión de reportes de incidencias y mantenimiento se realiza de manera manual, a través de oficios físicos, correos electrónicos dispersos y comunicación telefónica. Esto genera falta de trazabilidad, tiempos de respuesta prolongados, duplicidad de información, ausencia de estadísticas y asignación ineficiente de técnicos.

### Solución propuesta:

Se desarrolló EduInspect, un sistema web integral para la gestión de reportes de incidencias que permite: registrar reportes con ubicación, descripción y evidencias fotográficas; asignar automáticamente técnicos especializados; dar seguimiento en tiempo real al estado de cada incidencia; generar estadísticas y reportes de desempeño; notificar a los involucrados sobre cambios de estado; y mantener un historial completo de todas las acciones.

### Tecnologías seleccionadas:

|               |                  |                                                              |
|---------------|------------------|--------------------------------------------------------------|
| Capa          | Tecnología       | Justificación                                                |
| -----         | -----            | -----                                                        |
| Frontend      | Next.js 14       | Framework React con SSR, optimizado para rendimiento         |
| Backend       | FastAPI          | Alto rendimiento, tipado automático, documentación integrada |
| Base de Datos | PostgreSQL 16    | Confiabilidad, soporte JSON, cumplimiento ACID               |
| ORM           | Prisma           | Type-safe, migraciones automáticas                           |
| Autenticación | JWT + bcrypt     | Estándar industrial, seguro y sin estado                     |
| Contenedores  | Docker + Compose | Portabilidad, replicabilidad, fácil despliegue               |

### Beneficio esperado:

- Reducción del 70% en tiempos de respuesta a incidencias
- Trazabilidad completa de cada reporte
- Optimización de recursos humanos y materiales

- Generación automática de indicadores de desempeño

Alcance general:

El sistema incluye gestión completa de reportes (CRUD), catálogo de edificios y ubicaciones de la FES Aragón, 4 roles de usuario con permisos diferenciados, sistema de notificaciones, adjunto de imágenes, exportación de datos y despliegue mediante contenedores Docker.

---

## PUNTO 4: INTRODUCCIÓN

Contexto del problema:

La FES Aragón cuenta con más de 15,000 estudiantes, 500 profesores y 200 trabajadores administrativos distribuidos en múltiples edificios, laboratorios, bibliotecas y áreas deportivas. Esta infraestructura genera diariamente decenas de solicitudes de mantenimiento. Tradicionalmente, este flujo de trabajo se ha gestionado mediante oficios impresos, correos electrónicos no centralizados y llamadas telefónicas.

Durante el periodo 2024-2025, se identificó que aproximadamente el 40% de los reportes quedaban sin seguimiento y que el tiempo promedio entre la detección de una incidencia y su resolución superaba las 72 horas.

Importancia del sistema:

- Calidad educativa: Un entorno físico en mal estado afecta la experiencia de aprendizaje
- Eficiencia administrativa: Digitalizar reduce la carga de trabajo y elimina pérdida de documentos
- Transparencia: Estudiantes y autoridades pueden verificar el estado de cualquier reporte
- Toma de decisiones: Los datos generados permiten identificar patrones y asignar recursos inteligentemente

Digitalización:

| Proceso tradicional | Digitalización en EduInspect            |
|---------------------|-----------------------------------------|
| Oficio impreso      | Formulario web con campos estructurados |
| Correo electrónico  | Notificaciones automáticas push         |
| Asignación manual   | Selector de técnicos con historial      |

| Proceso tradicional     | Digitalización en EduInspect              |
|-------------------------|-------------------------------------------|
| Seguimiento por llamada | Tablero de control con estados            |
| Reporte en papel        | Dashboard con estadísticas en tiempo real |

Impacto administrativo:

- Nivel operativo: Inspectores reportan en <2 minutos; técnicos reciben asignaciones ordenadas
- Nivel táctico: Reportes semanales automáticos; identificación de áreas con recurrencia
- Nivel estratégico: Datos duros para presupuestos; mejora de imagen institucional

Objetivo general:

Desarrollar e implementar un sistema web integral para la gestión de reportes de incidencias en la FES Aragón - UNAM, que permita digitalizar el flujo de trabajo desde la detección hasta la resolución, optimizando los tiempos de respuesta, garantizando la trazabilidad de cada solicitud y proporcionando herramientas de análisis para la toma de decisiones administrativas.

## PUNTO 5: PLANTEAMIENTO DEL PROBLEMA

Situación actual:

El proceso actual tiene múltiples puntos de fricción: elaboración de oficio físico, registro en libro de control, envío a coordinación, reenvío a mantenimiento, asignación a técnico, reparación y reporte verbal.

Indicadores actuales:

| Indicador                            | Valor actual | Valor objetivo |
|--------------------------------------|--------------|----------------|
| Tiempo promedio de respuesta inicial | 8.5 horas    | 2 horas        |
| Tiempo promedio de resolución        | 72 horas     | 24 horas       |
| Reportes sin seguimiento             | 40%          | 5%             |
| Reportes duplicados                  | 25%          | 0%             |

Problemas detectados:

1. Falta de trazabilidad - Los oficios se extravían
2. Duplicidad de información - Un mismo problema reportado múltiples veces
3. Tiempos de respuesta prolongados - Burocracia excesiva
4. Ausencia de métricas - No hay datos confiables
5. Asignación ineficiente - No hay visibilidad de carga de trabajo
6. Pérdida de conocimiento histórico - Soluciones no documentadas
7. Comunicación fragmentada - Sin canal común
8. Dificultad para adjuntar evidencias - Sin repositorio centralizado

Impacto operativo:

- Inspectores: Desmotivación, pérdida de tiempo, frustración
- Técnicos: Desorganización, desplazamientos innecesarios, presión
- Coordinadores: Incapacidad para medir desempeño, sobrecarga administrativa
- Autoridades: Falta de indicadores, imposibilidad de identificar patrones

Riesgos:

| Riesgo                | Probabilidad | Impacto | Mitigación                |
|-----------------------|--------------|---------|---------------------------|
| Resistencia al cambio | Alta         | Medio   | Capacitación continua     |
| Pérdida de datos      | Media        | Alto    | Backup automatizado       |
| Caída del sistema     | Baja         | Alto    | Autoreinicio con Docker   |
| Vulnerabilidades JWT  | Baja         | Alto    | HTTPS, secret key robusta |

## PUNTO 6: OBJETIVOS

Objetivo general:

Desarrollar e implementar un sistema web integral para la gestión de reportes de incidencias en la FES Aragón - UNAM.

Objetivos específicos:

1. Diseñar una base de datos relacional que modele el flujo de trabajo de incidencias
2. Implementar un sistema de autenticación con 4 roles diferenciados

3. Desarrollar un panel de control con estadísticas en tiempo real
4. Crear un sistema de notificaciones automáticas por cambios de estado
5. Generar reportes exportables en formatos PDF y Excel
6. Permitir el adjunto de evidencias fotográficas
7. Documentar completamente el sistema
8. Garantizar la portabilidad mediante contenedores Docker

## PUNTO 7: MARCO TEÓRICO

### 7.1 Sistemas de Información

Un sistema de información es un conjunto de componentes interrelacionados que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones (Laudon & Laudon, 2020). Componentes: hardware, software, datos, procedimientos y recursos humanos.

### 7.2 Sistemas Administrativos

| Tipo | Función                | Ejemplo en EduInspect     |
|------|------------------------|---------------------------|
| TPS  | Procesar transacciones | Creación de reportes      |
| MIS  | Generar reportes       | Dashboard de estadísticas |
| DSS  | Apoyar decisiones      | Análisis de desempeño     |

### 7.3 Desarrollo Web

#### 7.3 Desarrollo Web

- Backend: FastAPI, Python 3.11

### 7.4 Bases de Datos

PostgreSQL 16, base de datos relacional con características avanzadas: tipos JSONB, índices parciales, cumplimiento ACID.

## 7.5 Incidencias y Mantenimiento

Clasificación por criticidad: Crítica (<4h), Alta (<24h), Media (<72h), Baja (<1sem).

## 7.6 Git y Control de Versiones

Sistema de control de versiones distribuido. Flujo Git Flow: main (producción), develop (integración), feature/ (nuevas características).

## 7.7 Docker y Contenedores

Plataforma de virtualización que empaqueta aplicaciones y dependencias en contenedores. EduInspect usa tres contenedores: PostgreSQL, Backend (FastAPI), Frontend (Next.js).

## 7.8 APIs REST

Estilo arquitectónico basado en recursos, métodos HTTP semánticos y stateless.

## 7.9 Autenticación y Seguridad

- JWT: Token con header, payload y signature
- bcrypt: Hash de contraseñas con salt
- CORS: Restricción de dominios

## 7.10 ORM (Prisma)

Traduce entre modelos de Python/TypeScript y tablas PostgreSQL. Type-safe, migraciones automáticas.

# PUNTO 8: ALCANCE DEL SISTEMA

Qué hará el sistema:

- Gestión de usuarios (4 roles)
- CRUD de reportes con ubicación, descripción y evaluación
- Adjunto de hasta 5 imágenes (5MB máximo)
- Asignación de técnicos

- Cambio de estado (pending → assigned → in\_progress → completed)
- Comentarios y bitácora de seguimiento
- Dashboard con indicadores clave
- Notificaciones push
- Exportación a PDF y Excel
- CRUD de edificios y ubicaciones
- Autenticación JWT
- Despliegue con Docker

Qué no hará el sistema:

- Aplicación móvil nativa
- Geolocalización en tiempo real
- Chat en vivo
- Integración con sistemas externos (SIASE, SAR)
- Órdenes de compra o facturación
- Modo offline
- Autenticación biométrica o 2FA
- Balanceo de carga automático

---

## PUNTO 9: REQUERIMIENTOS DEL SISTEMA

Requerimientos funcionales:

| ID    | Requerimiento                                                       |
|-------|---------------------------------------------------------------------|
| ----  | -----                                                               |
| RF-01 | El sistema debe permitir iniciar sesión con email y contraseña      |
| RF-02 | El sistema debe permitir crear reportes con ubicación y descripción |
| RF-03 | El sistema debe generar un número único de reporte (R-XXXXX)        |
| RF-04 | El sistema debe permitir cambiar el estado del reporte              |
| RF-05 | El sistema debe permitir asignar técnicos a reportes                |
| RF-06 | El sistema debe permitir agregar comentarios a reportes             |
| RF-07 | El sistema debe mostrar un dashboard con estadísticas               |
| RF-08 | El sistema debe permitir adjuntar imágenes como evidencia           |
| RF-09 | El sistema debe permitir gestionar usuarios (CRUD)                  |
| RF-10 | El sistema debe tener 4 roles con permisos diferentes               |
| RF-11 | El sistema debe mostrar notificaciones al usuario                   |
| RF-12 | El sistema debe permitir gestionar edificios y ubicaciones          |
| RF-13 | El sistema debe exportar reportes a PDF                             |
| RF-14 | El sistema debe cerrar sesión                                       |

Requerimientos no funcionales:

| ID | Requerimiento | Métrica |
|----|---------------|---------|
|----|---------------|---------|



| ID     | Regla                                        |
|--------|----------------------------------------------|
| RNF-01 | Rendimiento   Página principal <3 segundos   |
| RNF-02 | Seguridad   Contraseñas hasheadas con bcrypt |
| RNF-03 | Disponibilidad   99% tiempo activo           |
| RNF-04 | Usabilidad   Interfaz responsive             |
| RNF-05 | Portabilidad   Ejecutable con Docker         |
| RNF-06 | Mantenibilidad   Código documentado          |

Reglas de negocio:

| ID    | Regla                                                  |
|-------|--------------------------------------------------------|
| RN-01 | Solo inspectores y coordinadores pueden crear reportes |
| RN-02 | El número de reporte es autogenerado y único           |
| RN-03 | La asignación de técnico cambia estado a "assigned"    |
| RN-04 | Un reporte completado no puede reabrirse               |
| RN-05 | Solo administradores pueden eliminar usuarios          |
| RN-06 | La contraseña inicial por defecto es "Unam26!#"        |
| RN-07 | Las imágenes no pueden exceder 5MB                     |
| RN-08 | La ubicación debe pertenecer a un edificio existente   |

---

## PUNTO 10: CASOS DE USO

Actores del sistema:

| Actor         | Descripción                                      |
|---------------|--------------------------------------------------|
| Inspector     | Crea reportes y adjunta evidencias               |
| Coordinador   | Asigna técnicos, cambia estados, genera reportes |
| Técnico       | Atiende incidencias asignadas                    |
| Administrador | Gestiona usuarios y configuraciones              |

Diagrama de casos de uso:

[Insertar imagen: diagrama\_casos\_uso.png]

URL para generar: <https://mermaid.live/> (usar código proporcionado anteriormente)

Casos de uso principales:

- CU-01: Iniciar Sesión
- CU-02: Crear Reporte
- CU-03: Asignar Técnico
- CU-04: Cambiar Estado
- CU-05: Agregar Comentario

- CU-06: Ver Dashboard
- CU-07: Gestionar Personal
- CU-08: Adjuntar Evidencia
- CU-09: Ver Notificaciones
- CU-10: Gestionar Ubicaciones
- CU-11: Exportar PDF
- CU-12: Cerrar Sesión

---

## PUNTO 11: HISTORIAS DE USUARIO

### Historia 1 - Inspector:

"Como inspector de la FES Aragón, quiero poder reportar una incidencia con fotos y descripción, para que el equipo de mantenimiento conozca exactamente el problema y pueda resolverlo rápidamente."

#### Criterios de aceptación:

- Poder seleccionar edificio y ubicación desde un catálogo
- Adjuntar hasta 5 imágenes
- Recibir un número de folio al finalizar
- Ver el estado de mis reportes anteriores

### Historia 2 - Coordinador:

"Como coordinador de mantenimiento, quiero ver todos los reportes pendientes y poder asignarlos a técnicos especializados, para distribuir equitativamente la carga de trabajo."

#### Criterios de aceptación:

- Ver lista de reportes sin técnico asignado
- Ver carga actual de cada técnico
- Asignar reporte a técnico específico
- Recibir notificación de asignación

### Historia 3 - Técnico:

"Como técnico de mantenimiento, quiero recibir notificaciones cuando me asignen un reporte y poder actualizar su estado, para que todos sepan el progreso de la reparación."

#### Criterios de aceptación:

- Ver notificación de nueva asignación
- Ver lista de mis reportes asignados
- Cambiar estado a "en progreso" o "completado"
- Agregar comentarios y fotos de la reparación

#### Historia 4 - Administrador:

"Como administrador del sistema, quiero poder gestionar usuarios, edificios y ubicaciones, para mantener actualizada la información de la facultad."

#### Criterios de aceptación:

- Crear, editar y eliminar usuarios
- Activar o desactivar cuentas
- Gestionar catálogo de edificios
- Gestionar catálogo de ubicaciones

---

## PUNTO 12: TECNOLOGÍAS SELECCIONADAS

#### Backend:

| Tecnología | Versión | Propósito                |
|------------|---------|--------------------------|
| FastAPI    | 0.104.1 | Framework API REST       |
| Python     | 3.11    | Lenguaje de programación |
| Uvicorn    | 0.24.0  | Servidor ASGI            |
| Prisma     | 0.11.0  | ORM para PostgreSQL      |
| JWT        | -       | Autenticación stateless  |
| bcrypt     | 4.0.1   | Hash de contraseñas      |

#### Frontend:

| Tecnología   | Versión | Propósito       |
|--------------|---------|-----------------|
| Next.js      | 14.2.0  | Framework React |
| React        | 18.2.0  | Biblioteca UI   |
| TypeScript   | 5.3.3   | Tipado estático |
| Axios        | 1.7.9   | Cliente HTTP    |
| Tailwind CSS | 3.4.1   | Estilos CSS     |

#### Base de Datos:

| Tecnología | Versión | Propósito                |
|------------|---------|--------------------------|
| PostgreSQL | 16      | Base de datos relacional |
| Prisma     | 0.11.0  | ORM y migraciones        |

#### Infraestructura:

| Tecnología | Versión | Propósito |
|------------|---------|-----------|
|------------|---------|-----------|

|                |        |                      |
|----------------|--------|----------------------|
| -----          | -----  | -----                |
| Docker         | 20.10+ | Contenedores         |
| Docker Compose | 2.x    | Orquestación         |
| Git            | 2.x    | Control de versiones |
| GitHub         | -      | Repositorio remoto   |

Justificación técnica:

- FastAPI: Alto rendimiento, tipado automático, documentación integrada (Swagger)
- Next.js: SSR para mejor rendimiento, enrutamiento basado en archivos
- PostgreSQL: Confiabilidad, cumplimiento ACID, soporte nativo en Vercel/Neon
- Docker: Portabilidad total entre entornos, facilidad de despliegue
- Prisma: Type-safe, migraciones automáticas, excelente documentación

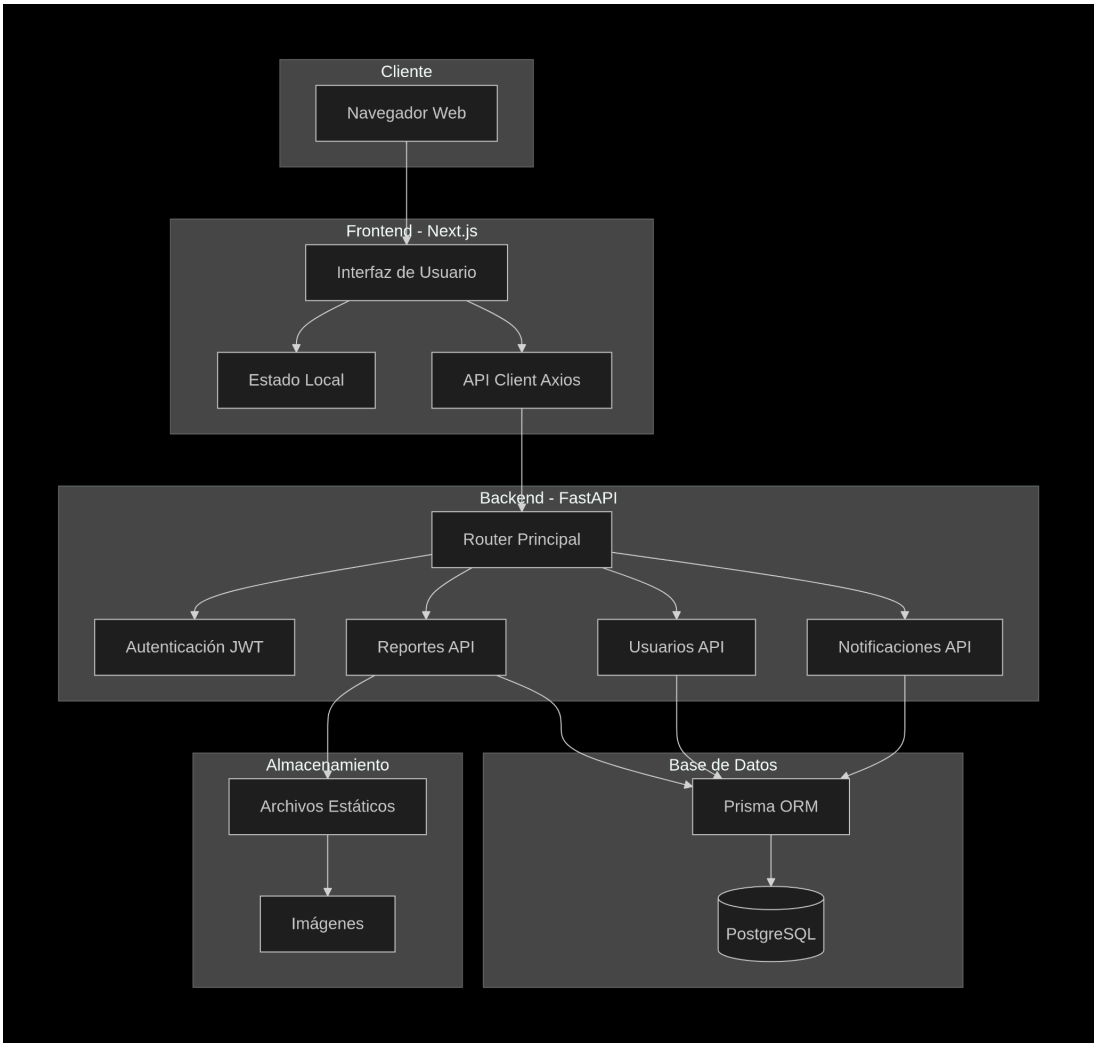
## PUNTO 13: ARQUITECTURA DEL SISTEMA

### 13.1 Arquitectura Cliente-Servidor

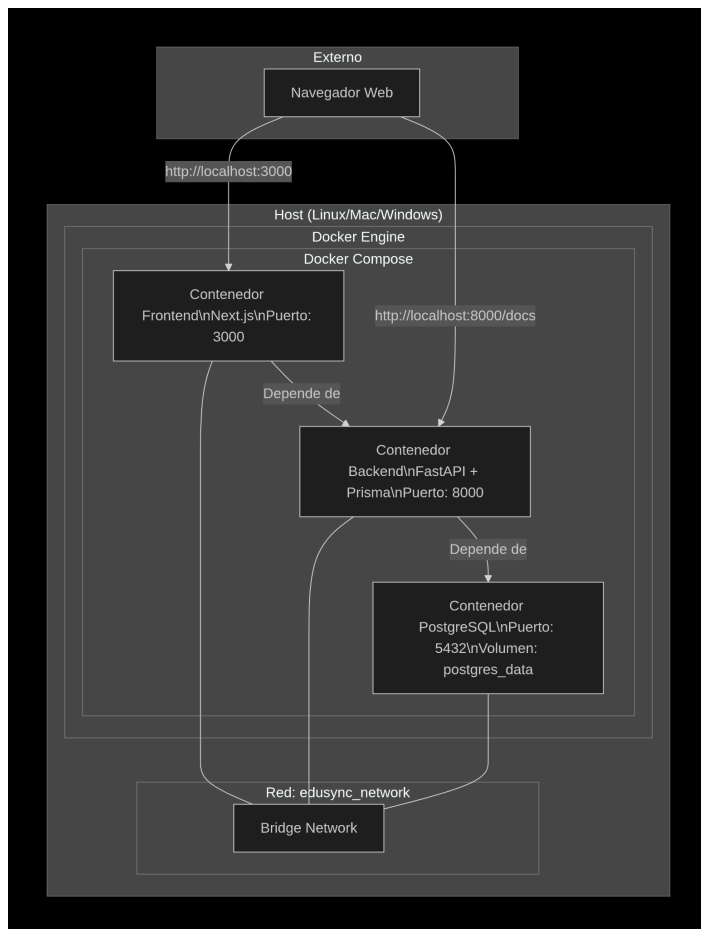
El sistema EduInspect implementa una arquitectura de tres capas:

|                   |               |               |                                                          |
|-------------------|---------------|---------------|----------------------------------------------------------|
| Capa              | Componente    | Tecnología    | Función                                                  |
| -----             | -----         | -----         | -----                                                    |
| Presentación      | Frontend      | Next.js 14    | Interfaz de usuario, interacción con el usuario          |
| Lógica de negocio | Backend       | FastAPI       | Procesamiento de datos, autenticación, reglas de negocio |
| Datos             | Base de Datos | PostgreSQL 16 | Almacenamiento persistente de información                |

### 13.2 Diagrama de Arquitectura del Sistema



## DIAGRAMA DESPLIEGE DOKER



## DIAGRAMA DE CLASES

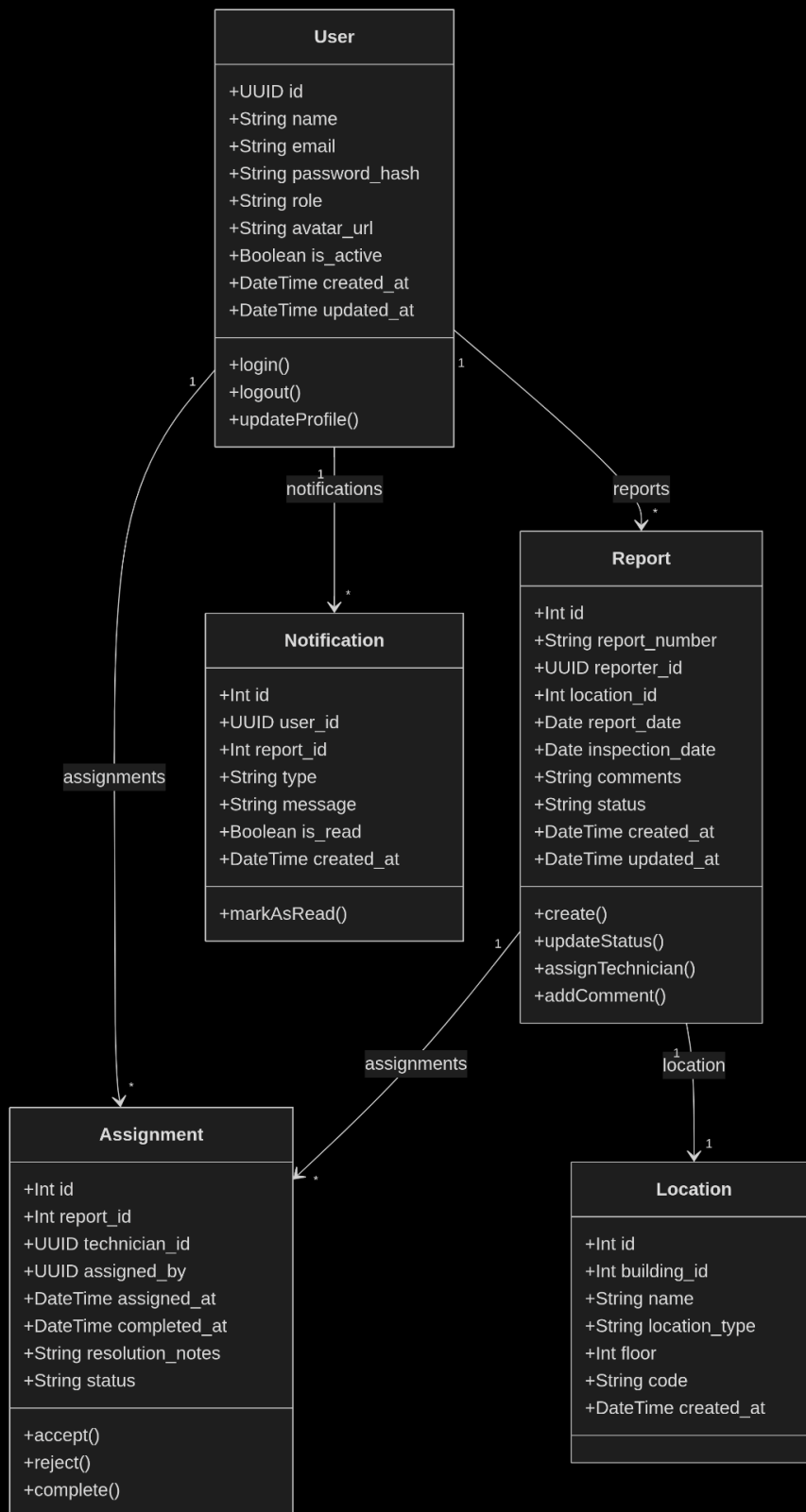


DIAGRAMA DE COMPONENTES

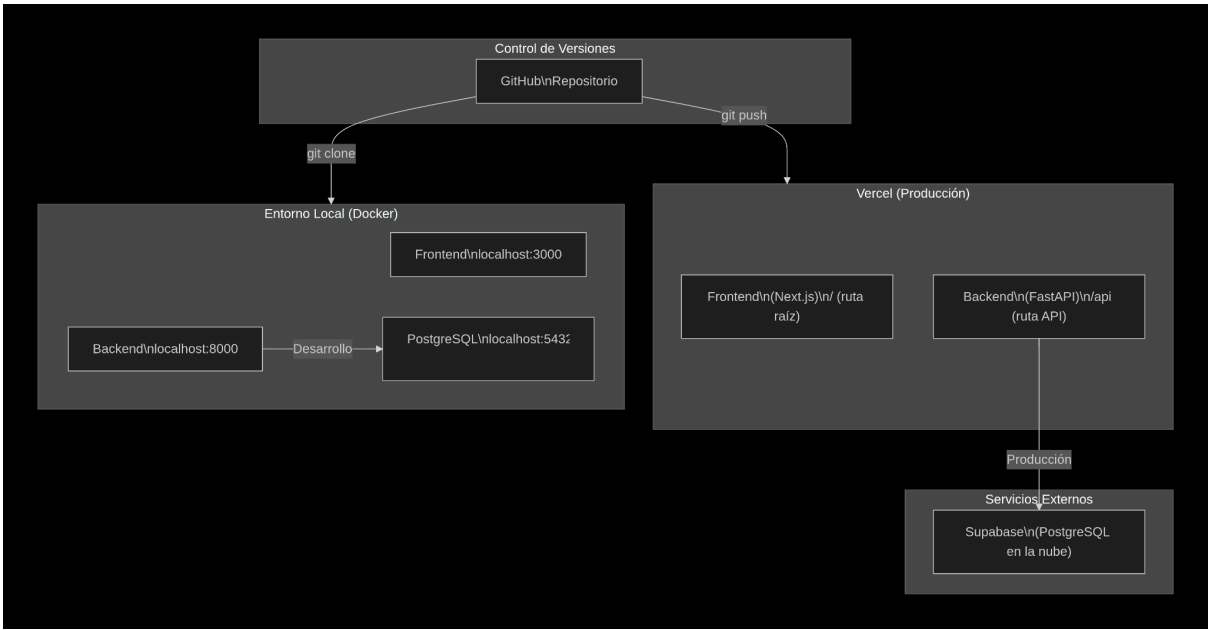
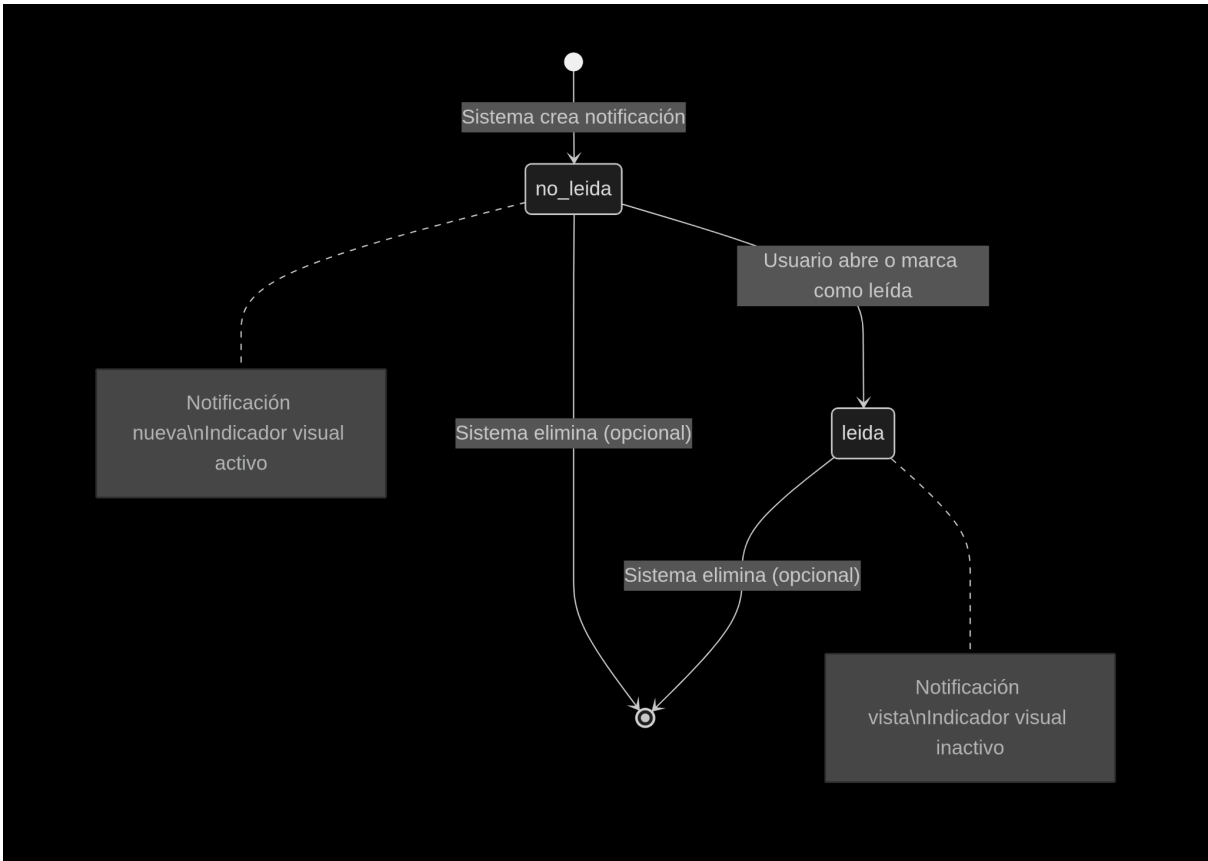
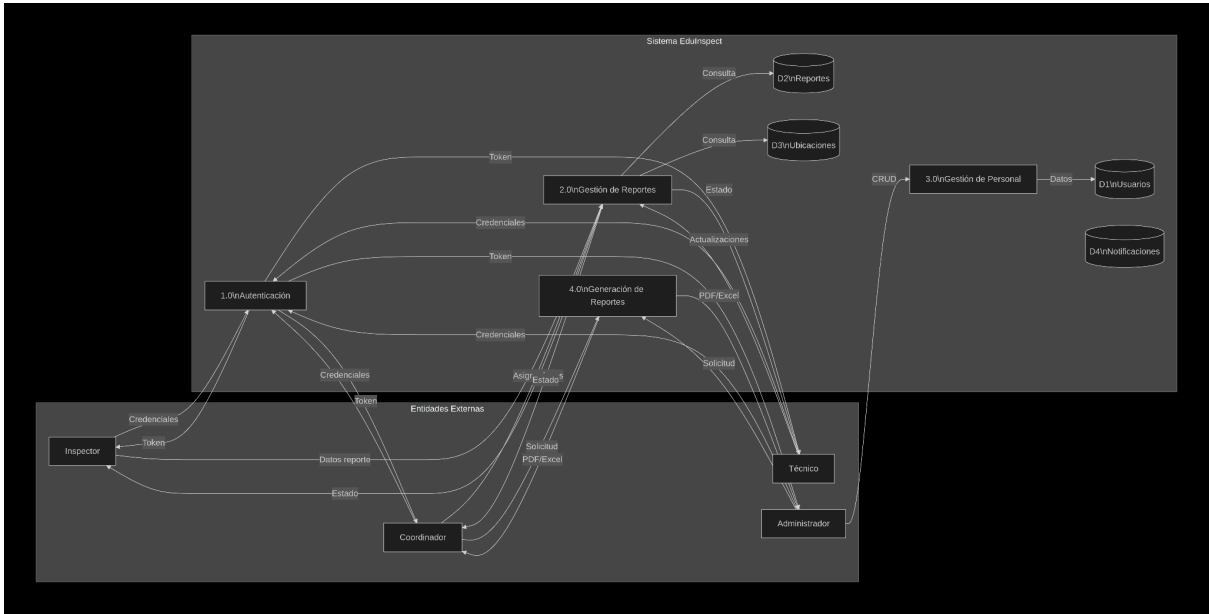


DIAGRAMA SISTREMA NOTIFICACIÓN

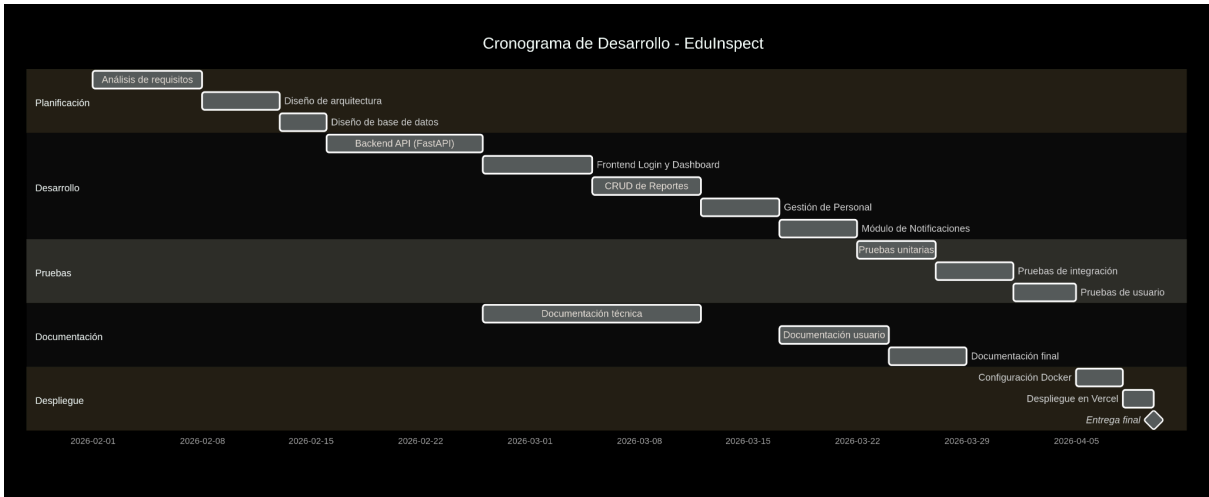




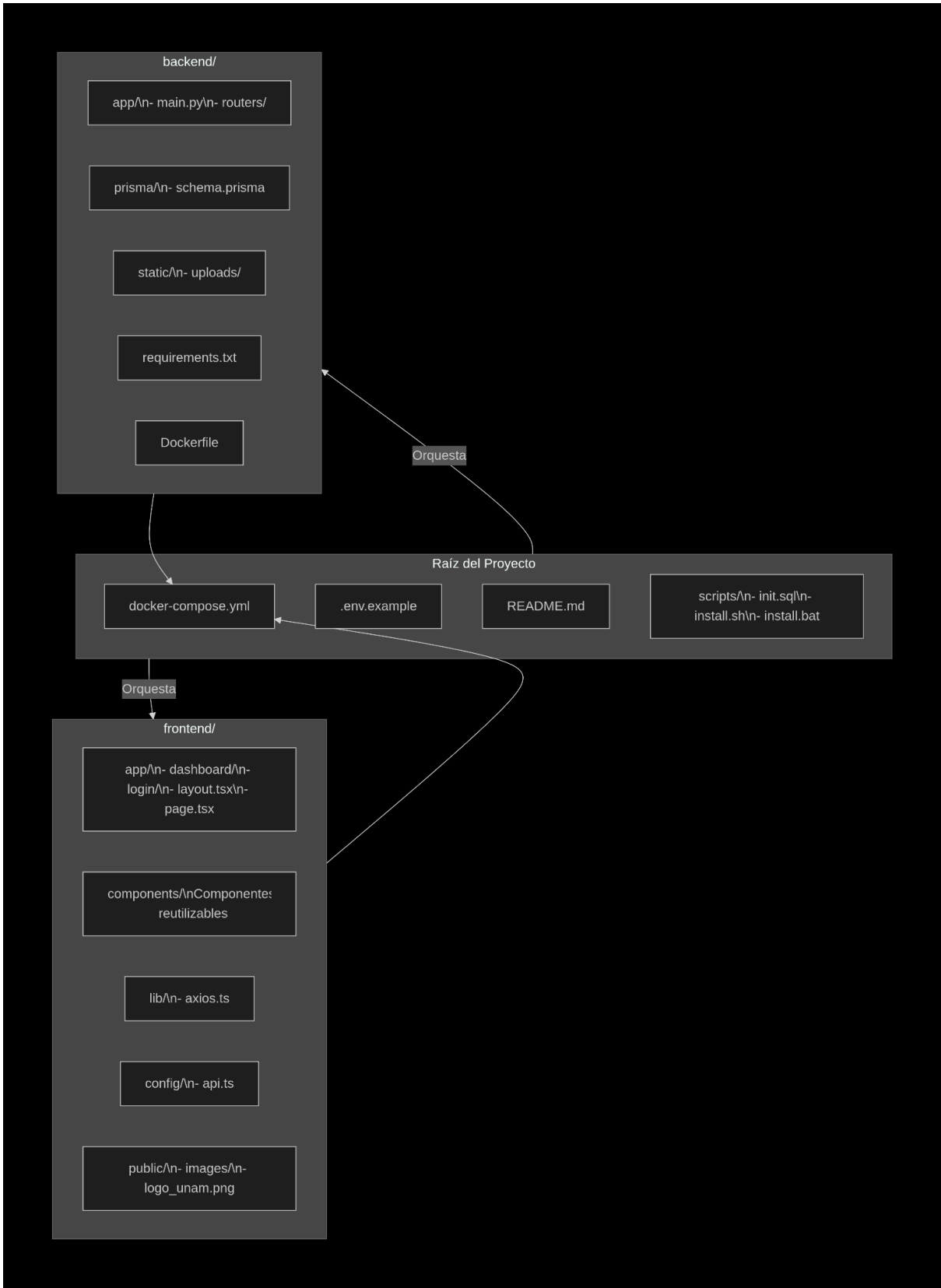
# DIAGRAMA DE FLUJO DE DATOS



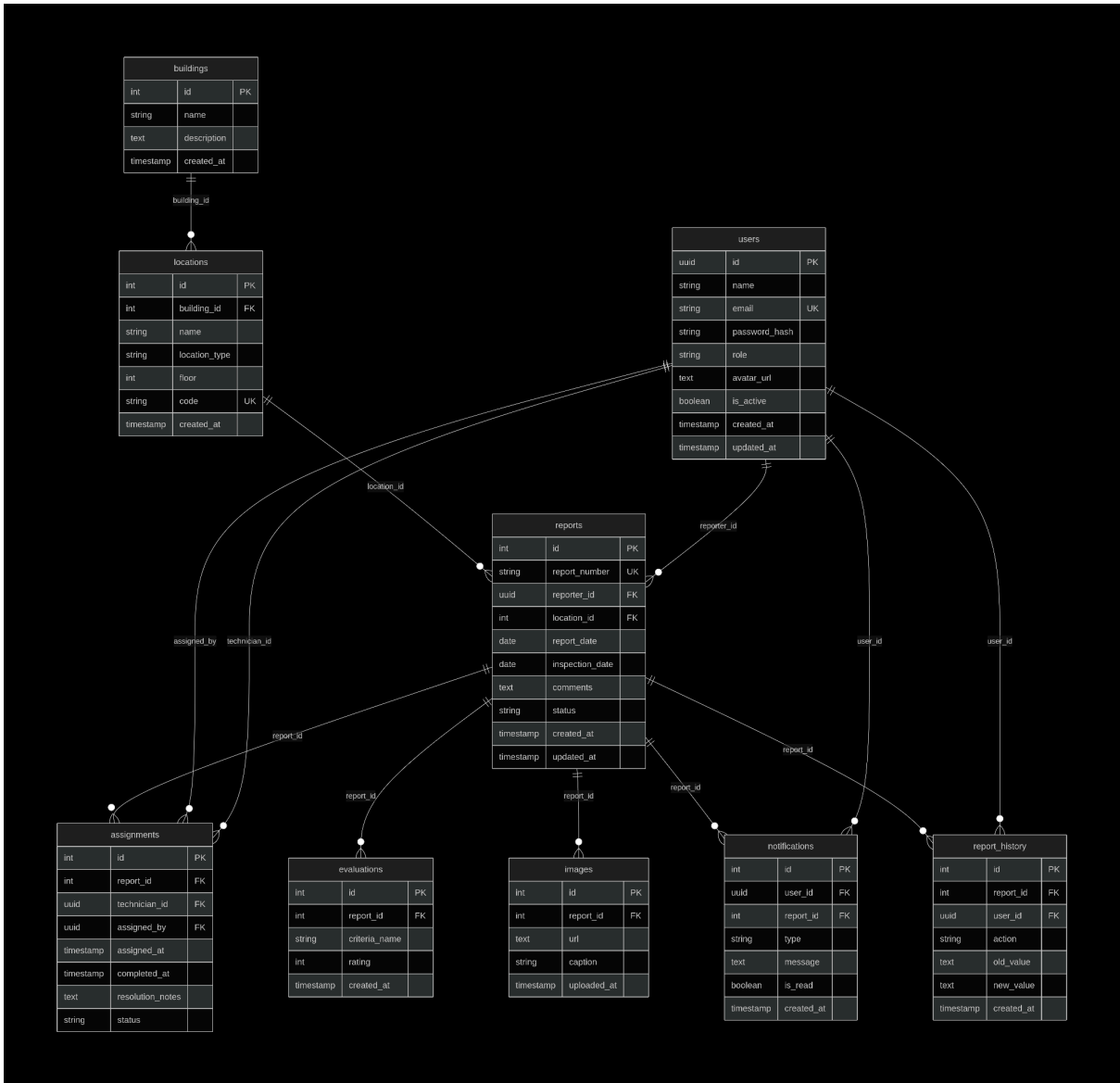
# DIAGRAMA DE GRANT



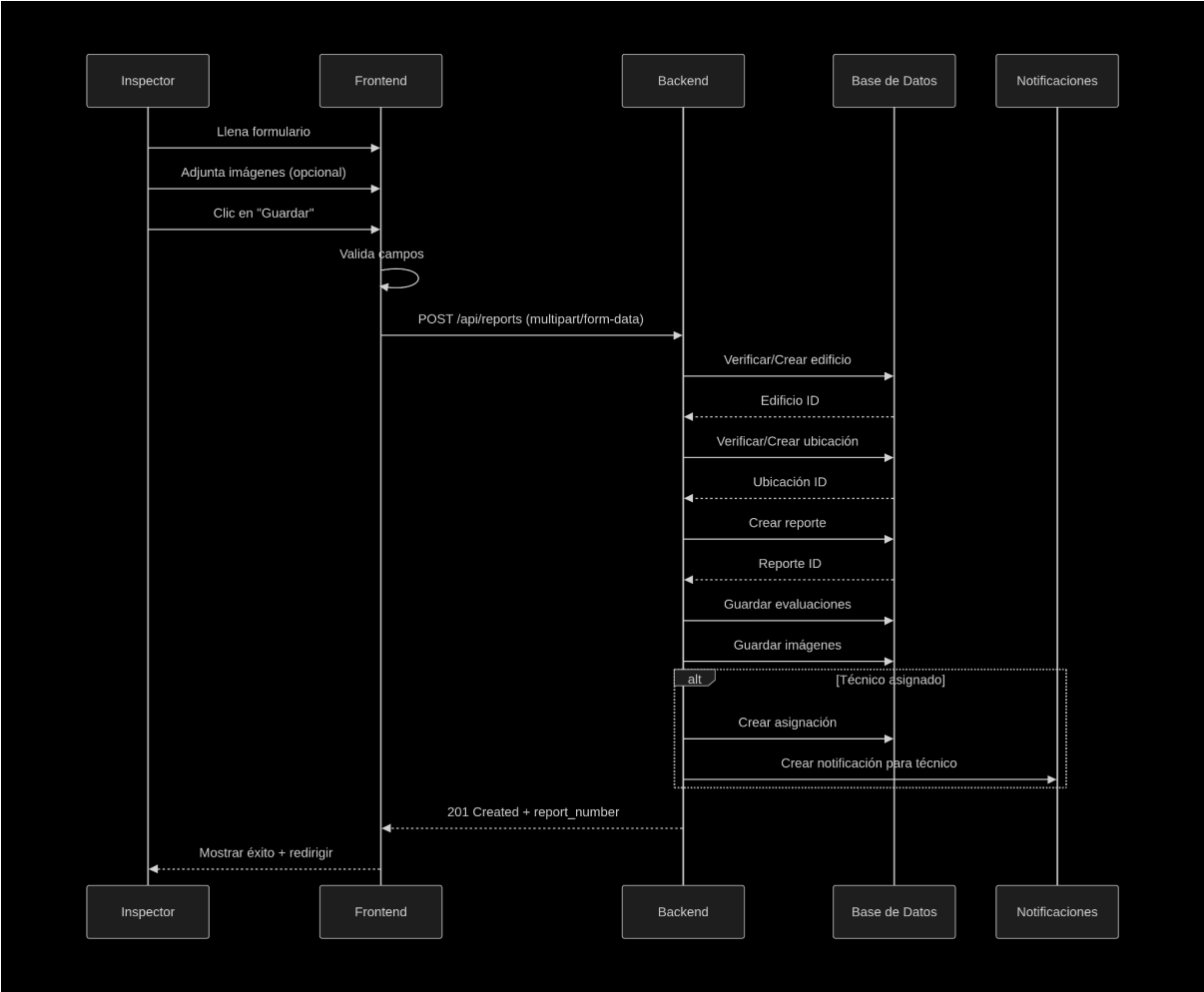
# DIAGRAMA DE PACKETES



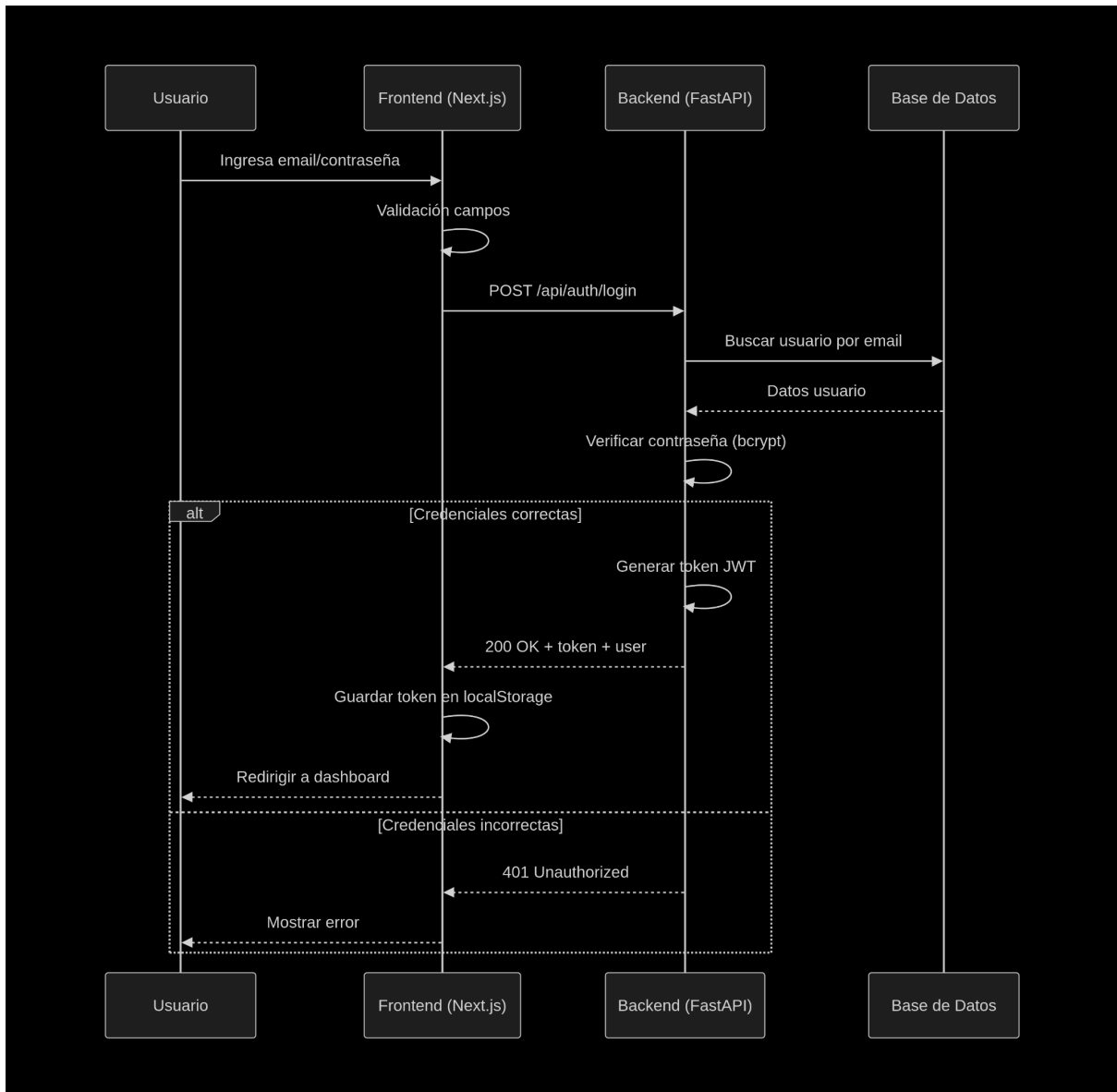
# DIAGRAMA ENTIDAD RELACION



# DIAGRAMA CREAR REPORTES



# DIAGRAMA LOGIN

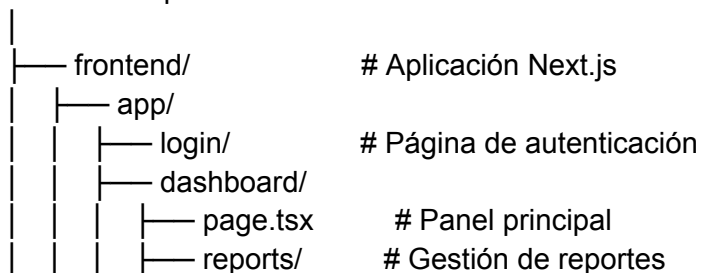


## PUNTO 14: ESTRUCTURA DEL PROYECTO

### 14.1 Organización de Carpetas

...

sistemadereportes/



|                        |                              |
|------------------------|------------------------------|
| ├── staff/             | # Gestión de personal        |
| ├── messages/          | # Notificaciones             |
| ├── layout.tsx         | # Layout principal           |
| ├── page.tsx           | # Landing page               |
| ├── globals.css        | # Estilos globales           |
| ├── components/        | # Componentes reutilizables  |
| ├── lib/               |                              |
| ├── axios.ts           | # Cliente HTTP               |
| ├── config/            |                              |
| ├── api.ts             | # Configuración de API       |
| ├── public/            |                              |
| ├── images/            | # Imágenes estáticas         |
| ├── package.json       | # Dependencias               |
| ├── Dockerfile         | # Docker para frontend       |
| ├── backend/           | # API FastAPI                |
| ├── app/               |                              |
| ├── main.py            | # Punto de entrada           |
| ├── routers/           | # Endpoints organizados      |
| ├── prisma/            |                              |
| ├── schema.prisma      | # Esquema de BD              |
| ├── static/            |                              |
| ├── uploads/           | # Imágenes subidas           |
| ├── requirements.txt   | # Dependencias Python        |
| ├── Dockerfile         | # Docker para backend        |
| ├── scripts/           |                              |
| ├── init.sql           | # Inicialización de BD       |
| ├── docker-compose.yml | # Orquestación Docker        |
| ├── .env.example       | # Variables de entorno       |
| ├── install.sh         | # Instalador Linux/Mac       |
| ├── install_mac.sh     | # Instalador Mac             |
| ├── install.bat        | # Instalador Windows         |
| ├── README.md          | # Documentación del proyecto |

## 14.2 Módulos y Responsabilidades

| Módulo        | Responsabilidad         | Archivos principales          |
|---------------|-------------------------|-------------------------------|
| Frontend      | UI, interacción usuario | app/dashboard/*, components/* |
| Backend       | API, lógica de negocio  | app/main.py, app/routers/*    |
| Base de Datos | Almacenamiento          | prisma/schema.prisma          |
| Autenticación | Login, JWT              | routers/auth.py               |
| Reportes      | CRUD incidencias        | routers/reports.py            |
| Personal      | Gestión usuarios        | routers/users.py              |

| Notificaciones | Alertas | routers/notifications.py |

---

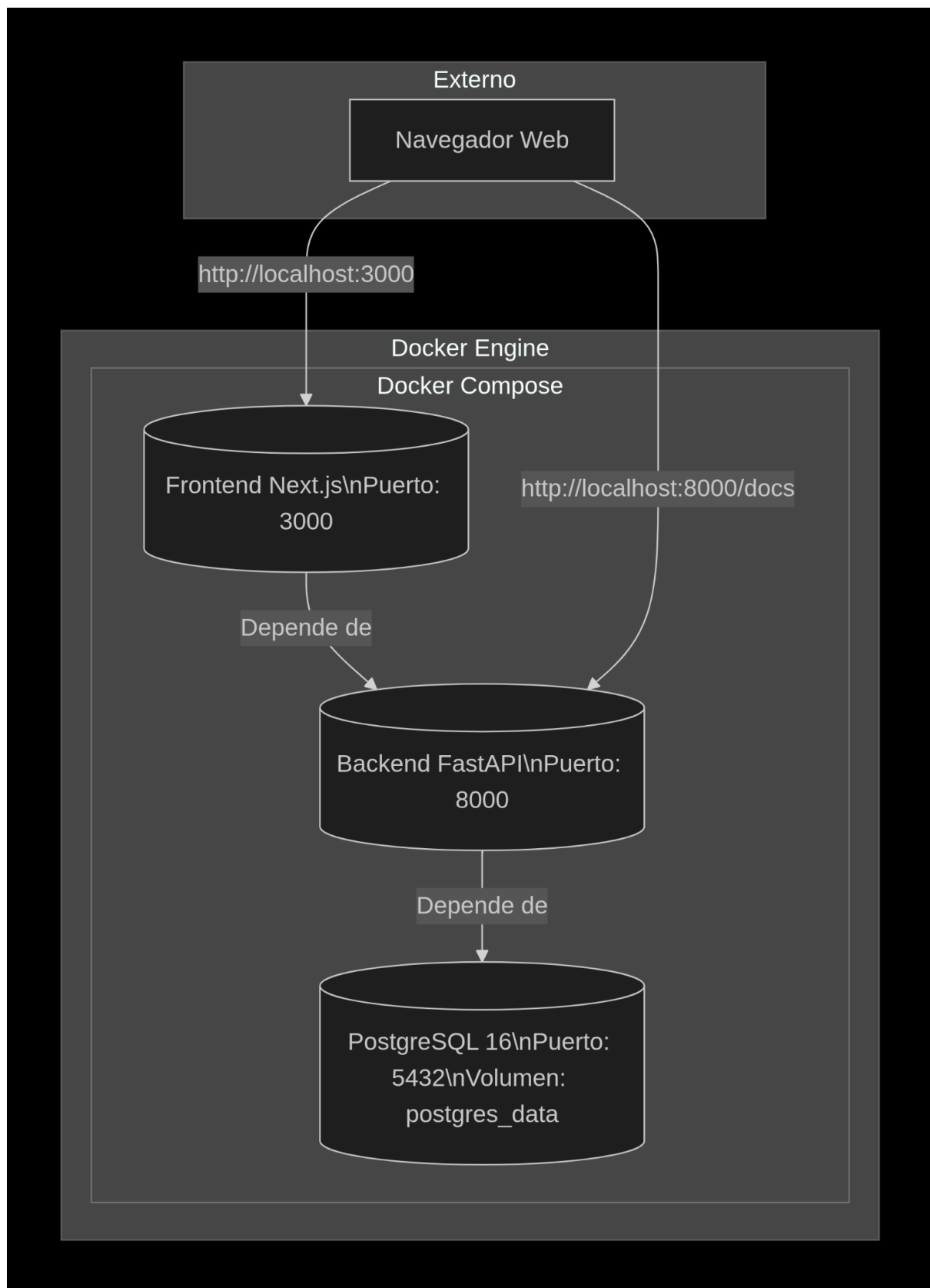
## PUNTO 15: USO DE CONTENEDORES Y DESPLIEGUE

### 15.1 Docker y Contenedores

Docker es una plataforma de contenedores que permite empaquetar aplicaciones con sus dependencias. EduInspect utiliza tres contenedores:

| Contenedor | Imagen             | Puerto | Dependencias |
|------------|--------------------|--------|--------------|
| PostgreSQL | postgres:16-alpine | 5432   | -            |
| Backend    | custom:backend     | 8000   | PostgreSQL   |
| Frontend   | custom:frontend    | 3000   | Backend      |

## 15.2 Diagrama de Contenedores Docker





## 15.3 Docker Compose (docker-compose.yml)

```
``yaml
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    container_name: edusync_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: eduinspect
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
    networks:
      - edusync_network

  backend:
    build: ./backend
    container_name: edusync_backend
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@postgres:5432/eduinspect
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - edusync_network

  frontend:
    build: ./frontend
    container_name: edusync_frontend
    ports:
      - "3000:3000"
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000
    depends_on:
      - backend
    networks:
      - edusync_network
```

```
volumes:
  postgres_data:

networks:
  edusync_network:
    driver: bridge
...

```

## 15.4 Comandos de Despliegue

| Comando                                        | Función                                          |
|------------------------------------------------|--------------------------------------------------|
| <code>`docker compose up -d`</code>            | Levantar todos los contenedores en segundo plano |
| <code>`docker compose down`</code>             | Detener todos los contenedores                   |
| <code>`docker compose down -v`</code>          | Detener y eliminar volúmenes (borra datos)       |
| <code>`docker compose logs -f`</code>          | Ver logs en tiempo real                          |
| <code>`docker compose build --no-cache`</code> | Reconstruir imágenes sin caché                   |
| <code>`docker compose restart`</code>          | Reiniciar todos los servicios                    |

## 15.5 Despliegue en Vercel (Producción)

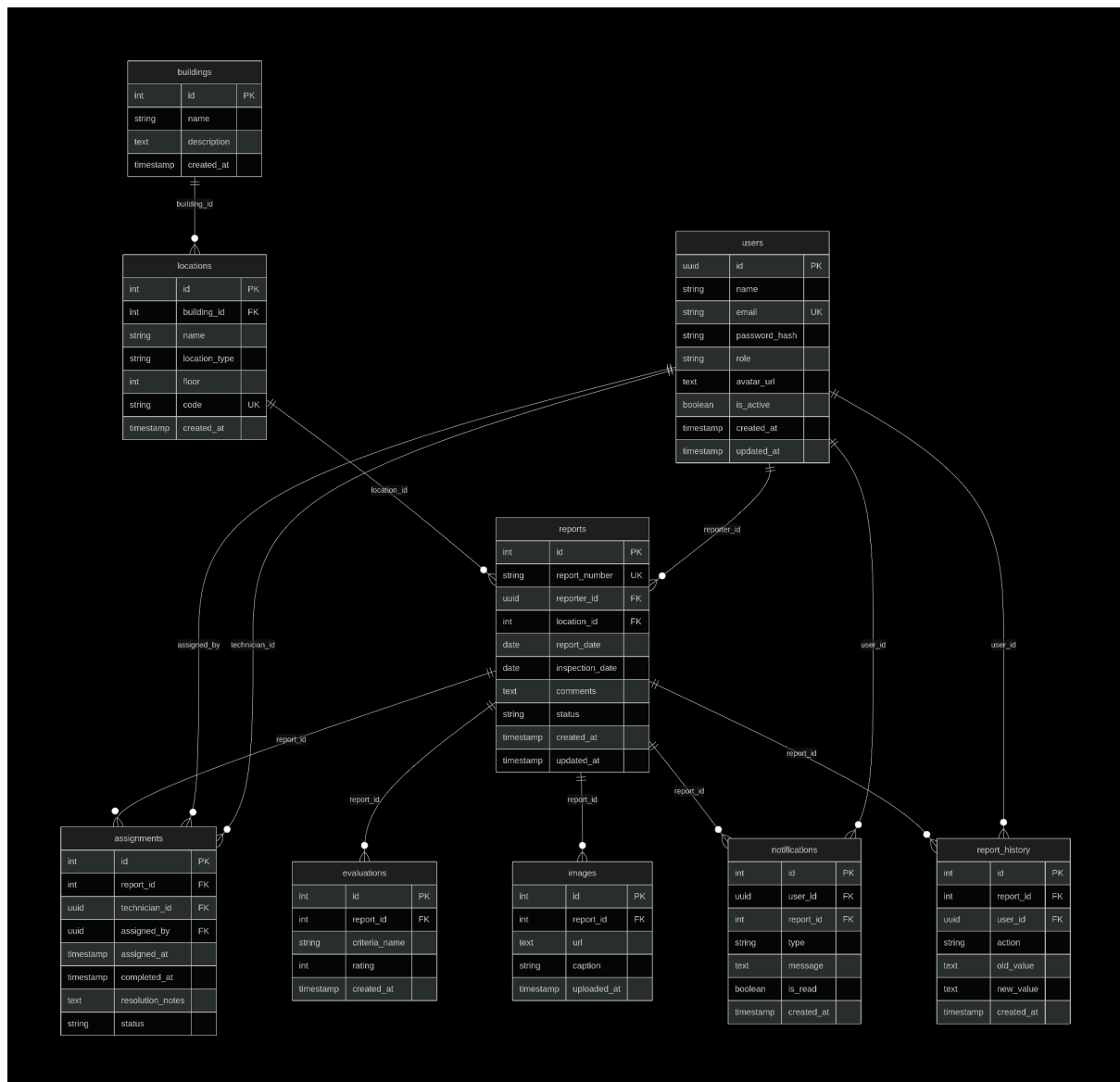
Para el despliegue en producción, se utiliza Vercel con las siguientes variables de entorno:

| Variable                           | Valor                       |
|------------------------------------|-----------------------------|
| <code>`DATABASE_URL`</code>        | PostgreSQL en Supabase/Neon |
| <code>`SECRET_KEY`</code>          | Clave secreta para JWT      |
| <code>`ALGORITHM`</code>           | HS256                       |
| <code>`NEXT_PUBLIC_API_URL`</code> | /api (relativo)             |

---

# PUNTO 16: MODELO DE BASE DE DATOS

## 16.1 Diagrama Entidad-Relación



## 16.2 Tablas y Relaciones

| Tabla | Descripción | Campos principales |

|-----|-----|-----|

| users | Usuarios del sistema | id, name, email, role, is\_active |

| buildings | Edificios de la facultad | id, name, description |

| locations | Ubicaciones dentro de edificios | id, building\_id, name, location\_type |

| reports | Reportes de incidencias | id, report\_number, reporter\_id, location\_id, status |

| evaluations | Calificaciones de reportes | id, report\_id, criteria\_name, rating |

| images | Evidencias fotográficas | id, report\_id, url, caption |

| assignments | Asignaciones de técnicos | id, report\_id, technician\_id, status |

| notifications | Notificaciones del sistema | id, user\_id, report\_id, type, is\_read |

| report\_history | Historial de cambios | id, report\_id, user\_id, action, new\_value |

## 16.3 Relaciones clave

- Un usuario puede tener muchos reportes (como reportero)
- Un usuario puede tener muchas asignaciones (como técnico)
- Un edificio puede tener muchas ubicaciones
- Una ubicación puede tener muchos reportes
- Un reporte puede tener muchas evaluaciones, imágenes, asignaciones, notificaciones y registros de historial

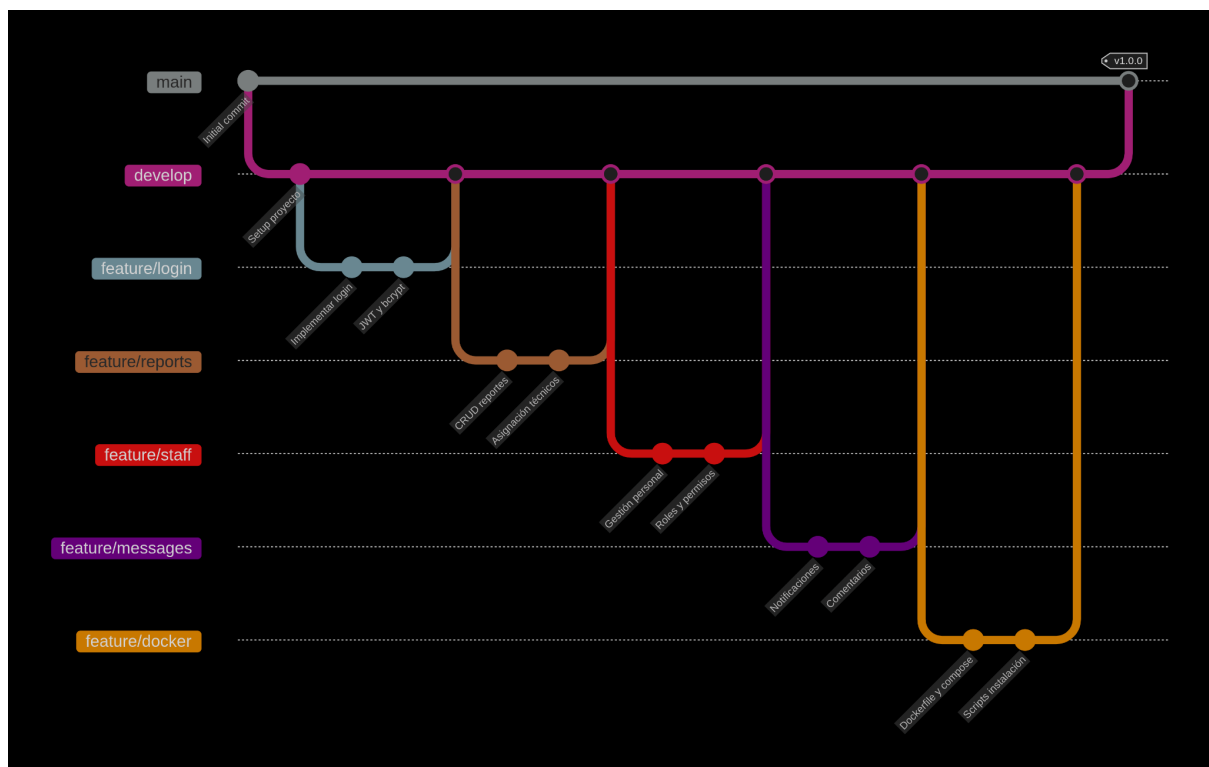
---

## PUNTO 17: ESTRATEGIA DE CONTROL DE VERSIONES

### 17.1 Git y GitHub

El proyecto utiliza Git para control de versiones y GitHub como repositorio remoto.

### 17.2 Flujo de trabajo (Git Flow)



## 17.3 Estructura de Ramas

| Rama                     | Propósito                      | Estado       |
|--------------------------|--------------------------------|--------------|
| ----- ----- -----        |                                |              |
| `main`                   | Código de producción estable   | ✓ Activa     |
| `dev`                    | Integración de características | ✓ Activa     |
| `feature/login-completo` | Autenticación JWT              | ✓ Completada |
| `feature/dashboard`      | Panel de control               | ✓ Completada |
| `feature/reportes`       | CRUD de reportes               | ✓ Completada |
| `feature/edificios`      | Gestión de ubicaciones         | ✓ Completada |
| `feature/Personal`       | Gestión de usuarios            | ✓ Completada |
| `feature/messages`       | Notificaciones                 | ✓ Completada |
| `feature/Docuemntación`  | Documentación                  | ✓ Completada |
| `fix/reports-crud`       | Correcciones CRUD              | ✓ Completada |
| `fix/report-comments`    | Correcciones comentarios       | ✓ Completada |

## 17.4 Convención de Commits

| Tipo              | Formato                 | Ejemplo                             |
|-------------------|-------------------------|-------------------------------------|
| ----- ----- ----- |                         |                                     |
| feat              | `feat: descripción`     | `feat: agregar login con JWT`       |
| fix               | `fix: descripción`      | `fix: corregir error en asignación` |
| docs              | `docs: descripción`     | `docs: actualizar README`           |
| style             | `style: descripción`    | `style: formatear código`           |
| refactor          | `refactor: descripción` | `refactor: optimizar consultas`     |
| test              | `test: descripción`     | `test: agregar pruebas unitarias`   |

## 17.5 Commits principales del proyecto

| Commit      | Descripción                       |
|-------------|-----------------------------------|
| ----- ----- |                                   |
| `3da96f7`   | Sistema funcionando al 100%       |
| `01d428e`   | Configuración Tailwind v3         |
| `c310eb7`   | Remover Tailwind, usar CSS plano  |
| `39f495e`   | Corregir dependencias para Vercel |

---

## PUNTO 20: RIESGOS DEL PROYECTO

## 20.1 Riesgos Técnicos

| Riesgo                       | Probabilidad | Impacto | Mitigación                                  |
|------------------------------|--------------|---------|---------------------------------------------|
| Vulnerabilidades JWT         | Baja         | Alto    | HTTPS, secret key robusta, expiración corta |
| Incompatibilidad navegadores | Baja         | Medio   | Pruebas en Chrome, Firefox, Edge, Safari    |
| Saturación BD                | Baja         | Medio   | Índices optimizados, pruning programado     |
| Fallo servidor BD            | Baja         | Alto    | Backups diarios, replicación en producción  |
| Error en bcrypt/passlib      | Media        | Alto    | Versiones específicas (bcrypt==4.0.1)       |
| Dependencias desactualizadas | Media        | Bajo    | `npm audit` y `pip list --outdated`         |

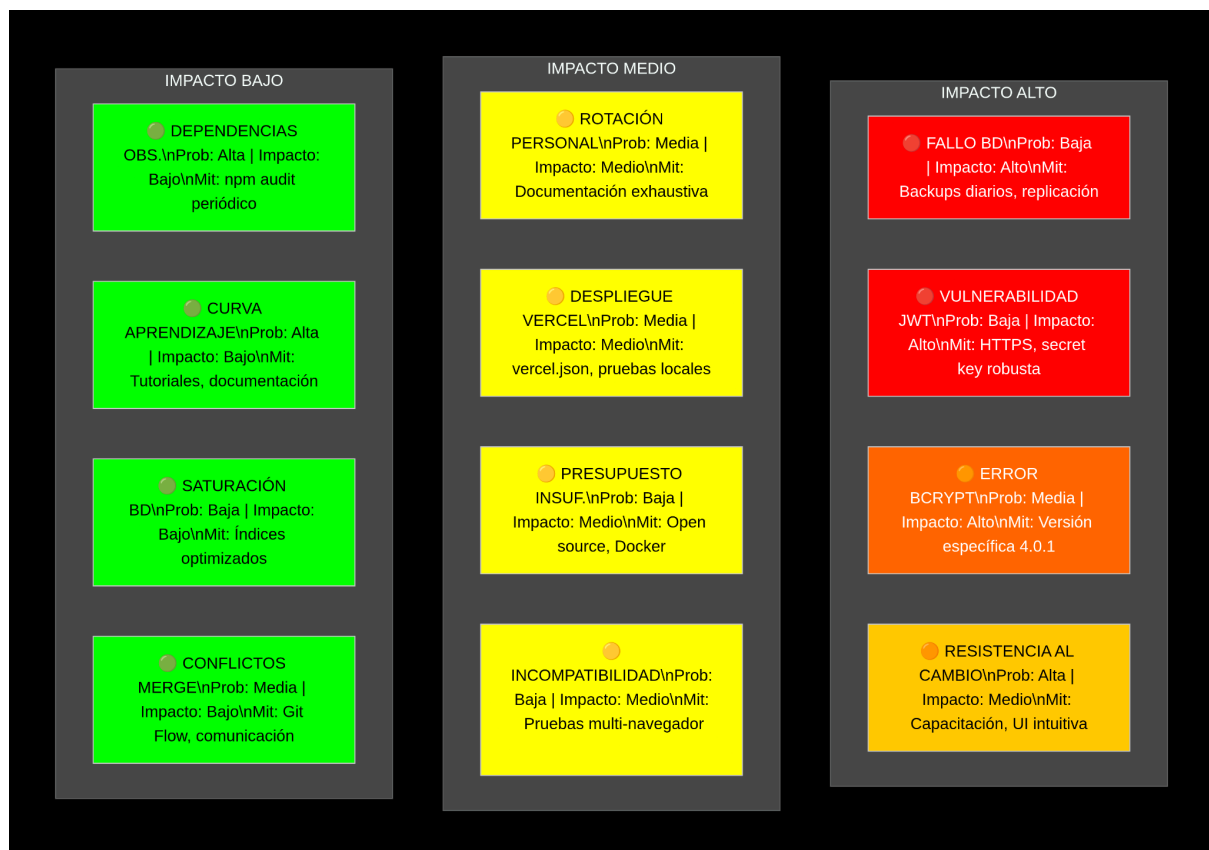
## 20.2 Riesgos Organizacionales

| Riesgo                          | Probabilidad | Impacto | Mitigación                       |
|---------------------------------|--------------|---------|----------------------------------|
| Resistencia al cambio           | Alta         | Medio   | Capacitación, interfaz intuitiva |
| Rotación de personal            | Media        | Medio   | Documentación exhaustiva         |
| Falta de respaldo institucional | Baja         | Alto    | Evidencias de beneficios         |
| Presupuesto insuficiente        | Baja         | Medio   | Tecnologías open source, Docker  |

## 20.3 Riesgos de Desarrollo

| Riesgo               | Probabilidad | Impacto | Mitigación                                 |
|----------------------|--------------|---------|--------------------------------------------|
| Curva de aprendizaje | Media        | Bajo    | Documentación, tutoriales                  |
| Conflictos de merge  | Media        | Bajo    | Git Flow, comunicación en equipo           |
| Problemas con Docker | Baja         | Alto    | Docker Compose, scripts automatizados      |
| Despliegue en Vercel | Media        | Medio   | Configuración vercel.json, pruebas locales |

## 20.4 Matriz de Riesgos



## PUNTO 21: CONCLUSIONES

### 21.1 Beneficios del Sistema

- Para inspectores: Reducción de tiempo de reporte de 30 minutos a 2 minutos
- Para coordinadores: Visibilidad completa de la carga de trabajo
- Para técnicos: Asignaciones ordenadas con prioridad y ubicación
- Para autoridades: Datos duros para planeación y presupuestos
- Para la comunidad: Canal oficial de reporte con trazabilidad

### 21.2 Aprendizajes Esperados

- Técnicos:
  - Desarrollo full-stack con Next.js y FastAPI
  - Contenerización con Docker y Docker Compose
  - Despliegue en Vercel y Supabase
  - Control de versiones con Git Flow

- Metodológicos:
  - Gestión de proyectos ágil
  - Documentación como parte integral del desarrollo
  - Pruebas y control de calidad
- Organizacionales:
  - Importancia de la digitalización de procesos
  - Impacto de la tecnología en la eficiencia administrativa
  - Necesidad de interfaces intuitivas

## 21.3 Resultados Alcanzados

| Objetivo                   | Estado       |
|----------------------------|--------------|
| Base de datos relacional   | ✓ Completado |
| Autenticación con 4 roles  | ✓ Completado |
| Dashboard con estadísticas | ✓ Completado |
| Sistema de notificaciones  | ✓ Completado |
| Exportación PDF/Excel      | ✓ Completado |
| Adjunto de evidencias      | ✓ Completado |
| Documentación completa     | ✓ Completado |
| Portabilidad con Docker    | ✓ Completado |

## 21.4 Trabajo Futuro

- Aplicación móvil nativa (React Native)
- Geolocalización en tiempo real
- Integración con SIASE y correo institucional
- Chat en vivo con WebSockets
- Reportes predictivos con machine learning
- Dashboard con más métricas y gráficas

---

## PUNTO 22: REFERENCIAS

Formato APA

Libros y documentos técnicos:

1. Laudon, K. C., & Laudon, J. P. (2020). \*Sistemas de información gerencial\* (14ª ed.). Pearson Educación.



2. Pressman, R. S. (2019). \*Ingeniería de software: Un enfoque práctico\* (9ª ed.). McGraw-Hill.

3. Sommerville, I. (2018). \*Ingeniería de software\* (10ª ed.). Pearson Educación.

Documentación oficial:

4. Vercel. (2025). \*Vercel Documentation - Deploying Next.js applications\*. Recuperado de <https://vercel.com/docs>

5. FastAPI. (2025). \*FastAPI Documentation\*. Recuperado de <https://fastapi.tiangolo.com>

6. Next.js. (2025). \*Next.js Documentation\*. Recuperado de <https://nextjs.org/docs>

7. PostgreSQL. (2025). \*PostgreSQL 16 Documentation\*. Recuperado de <https://www.postgresql.org/docs/16>

8. Docker. (2025). \*Docker Documentation\*. Recuperado de <https://docs.docker.com>

9. Prisma. (2025). \*Prisma ORM Documentation\*. Recuperado de <https://www.prisma.io/docs>

10. Supabase. (2025). \*Supabase Documentation - PostgreSQL Database\*. Recuperado de <https://supabase.com/docs>

Recursos adicionales:

11. GitHub. (2025). \*GitHub Documentation - Git Flow\*. Recuperado de <https://docs.github.com>

12. Tailwind CSS. (2025). \*Tailwind CSS Documentation\*. Recuperado de <https://tailwindcss.com/docs>

13. Axios. (2025). \*Axios Documentation\*. Recuperado de <https://axios-http.com/docs>

14. JWT.io. (2025). \*JSON Web Tokens Introduction\*. Recuperado de <https://jwt.io/introduction>

15. Bcrypt. (2025). \*bcrypt - Password hashing function\*. Recuperado de <https://pypi.org/project/bcrypt/>

---

ANEXOS (EXTRAS)

Matriz de Trazabilidad de Requerimientos

| Requerimiento | CU-01 | CU-02 | CU-03 | CU-04 | CU-05 | CU-06 | CU-07 | CU-08 | CU-09 |  
CU-10 | CU-11 | CU-12 |

|       | CU-01 | CU-02 | CU-03 | CU-04 | CU-05 | CU-06 | CU-07 | CU-08 | CU-09 | CU-10 | CU-11 | CU-12 |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| RF-01 | ✓     |       |       |       |       |       |       |       |       |       |       |       |
| RF-02 |       | ✓     |       |       |       |       |       |       |       |       |       |       |
| RF-03 |       | ✓     |       |       |       |       |       |       |       |       |       |       |
| RF-04 |       |       | ✓     |       |       |       |       |       |       |       |       |       |
| RF-05 |       |       | ✓     |       |       |       |       |       |       |       |       |       |
| RF-06 |       |       |       | ✓     |       |       |       |       |       |       |       |       |
| RF-07 |       |       |       |       | ✓     |       |       |       |       |       |       |       |
| RF-08 |       | ✓     |       |       |       | ✓     |       |       |       |       |       |       |
| RF-09 |       |       |       |       | ✓     |       |       |       |       |       |       |       |
| RF-10 |       |       |       |       | ✓     |       |       |       |       |       |       |       |
| RF-11 |       |       |       |       |       | ✓     |       |       |       |       |       |       |
| RF-12 |       |       |       |       |       | ✓     |       |       |       |       |       |       |
| RF-13 |       |       |       |       |       |       | ✓     |       |       |       |       |       |
| RF-14 |       |       |       |       |       |       |       | ✓     |       |       |       |       |

#### Evidencias de GitHub

Repositorio: <https://github.com/piaramirez/sistemadereportes>

#### Commits principales:

- `3da96f7` - Sistema funcionando al 100%
- `01d428e` - Configuración Tailwind v3
- `c310eb7` - Remover Tailwind, usar CSS plano
- `39f495e` - Corregir dependencias para Vercel

---